



# Proyecto final integrador

Analítica a gran escala y benchmarking con Apache Spark y Pandas

**Prof. Ing. Matías A. Pérez**

Maestría en Inteligencia Artificial — Asignatura: Big Data — Universidad de Palermo

## Planificación articulada entre clase 7 y clase 8

### Clase 7: Lanzamiento y preparación

- Presentación oficial del proyecto y pautas de entrega.
- Conocimiento del dataset (US Accidents).
- Confirmación de grupos y asignación de consultas.
- Inicio temprano de la descarga y exploración preliminar de datos.

### Clase 8: Taller práctico en clase

- Trabajo intensivo en clase dedicado al proyecto.
- Implementación de pipelines en Pandas y PySpark.
- Acompañamiento docente y resolución de consultas técnicas en vivo.
- Medición de tiempos de cómputo y benchmarking.

### Instancia final

Entrega del artículo científico (Formato\_Investigadores.doc) y defensa oral ante el docente.

# El dataset: US Accidents (2016 – 2023)



## Datos masivos reales de siniestralidad vial en Estados Unidos

### Magnitud de los datos

- **Volumen:**  $\sim 7.728.394$  registros continuos.
- **Variables:** 46 atributos espacio-temporales y contextuales.
- **Tamaño:**  $\sim 3.0$  GB en archivo plano CSV.
- **Alcance:** 49 estados continentales de EE.UU.

### Desafío de escala

Desborda la capacidad de herramientas de oficina convencionales y pone a prueba la gestión de memoria en Python mononodo.

### Principales atributos de análisis

- **Tiempo:** Start\_Time, End\_Time.
- **Ubicación:** Start\_Lat, Start\_Lng, End\_Lat, End\_Lng, Street, City, State.
- **Clima:** Temperature, Humidity, Visibility, Precipitation, Weather\_Condition.
- **Infraestructura:** Crossing, Junction, Railway, Traffic\_Signal.
- **Gravedad:** Severity (niveles 1 a 4).

# El núcleo del desafío: Pandas vs. Apache Spark

## Benchmarking comparativo de escalabilidad y límites de cómputo

### Python con Pandas (mononodo)

- Cómputo en memoria centralizada (RAM).
- Excelente ergonomía en volúmenes pequeños/medianos.
- Con millones de registros: saturación de memoria, paginación en disco y riesgo inminente de **Out Of Memory (OOM)**.

### Python con Apache Spark (PySpark)

- Procesamiento distribuido y particionado en clúster/núcleos.
- Evaluación perezosa y optimización lógica/física (*Catalyst*).
- Generación de código nativo en memoria (*Project Tungsten*).
- Escalabilidad horizontal ante grandes volúmenes.

### Requerimiento de benchmarking

Cada consulta debe cronometrarse en ambas tecnologías (*wall-clock time*), calculando el factor de aceleración:

$$\text{Speedup} = \frac{T_{\text{Pandas}}}{T_{\text{Spark}}}.$$

# Catálogo analítico: 10 consultas diseñadas



## 5 consultas por grupo combinando análisis espacial, temporal y multivariable

### ★ Consulta 4 [común a todos los 7 grupos]

#### Hotspots espaciales: top 5 puntos más peligrosos

Identificar las 5 coordenadas más críticas definiendo un radio geográfico de influencia ( $R = 2.0$  km o  $5.0$  km). Exige definir formalmente el criterio de peligrosidad (densidad, siniestros graves o índice ponderado) y justificar la técnica de agregación espacial empleada.

### ☰ Ámbitos de las consultas complementarias (C1 a C10)

- **Temporales:** Filtro entre fechas y distribución horaria (C1); despeje diurno vs. nocturno (C10).
- **Geoespaciales:** Radio geográfico con fórmula de Haversine (C3); distancia/duración inicio-fin (C5).
- **Contextuales:** Condiciones más comunes (C2); impacto de infraestructura vial en severidad (C7).
- **Analítica avanzada:** Ranking de autopistas (C6); anomalías climáticas  $> 3\sigma$  (C8); minería de texto en cruces (C9).

# Asignación de consultas por grupo

## Distribución equilibrada de 5 consultas para los 7 equipos

| Grupo   | Obligatoria           | Consulta A | Consulta B | Consulta C | Consulta D  |
|---------|-----------------------|------------|------------|------------|-------------|
| Grupo 1 | Consulta 4 (Hotspots) | Consulta 1 | Consulta 2 | Consulta 6 | Consulta 7  |
| Grupo 2 | Consulta 4 (Hotspots) | Consulta 2 | Consulta 3 | Consulta 5 | Consulta 8  |
| Grupo 3 | Consulta 4 (Hotspots) | Consulta 1 | Consulta 5 | Consulta 9 | Consulta 10 |
| Grupo 4 | Consulta 4 (Hotspots) | Consulta 3 | Consulta 6 | Consulta 8 | Consulta 10 |
| Grupo 5 | Consulta 4 (Hotspots) | Consulta 1 | Consulta 3 | Consulta 7 | Consulta 9  |
| Grupo 6 | Consulta 4 (Hotspots) | Consulta 2 | Consulta 5 | Consulta 6 | Consulta 8  |
| Grupo 7 | Consulta 4 (Hotspots) | Consulta 3 | Consulta 5 | Consulta 7 | Consulta 10 |

### ✓ Alcance por equipo

Cada equipo responde exactamente 5 consultas, ejecutando ambas implementaciones (Pandas y PySpark) y contrastando sus resultados.

# Consigna 6: Machine learning a escala con Spark MLlib



Aplicación distribuida con `pyspark.ml` a libre elección del grupo

## Tres alternativas (elegir solo una)

- **Opción A — Clasificación:** Predicción discreta (ej. siniestro grave con  $\text{Severity} \geq 3$ ). Modelos: `LogisticRegression`, `RandomForest`. Métricas: ROC-AUC / F1.
- **Opción B — Regresión:** Predicción continua (ej. duración del siniestro en minutos). Modelos: `LinearRegression`, `GBTRRegressor`. Métricas: RMSE,  $R^2$ .
- **Opción C — Clusterización:** Segmentación no supervisada (espacio-clima). Modelo: `KMeans`. Métrica: Silueta (*Silhouette*).

## Requisitos metodológicos

1. **Justificación:** Formular hipótesis y variables clave.
2. **Pipeline `pyspark.ml`:** `VectorAssembler`, nulos y escalado.
3. **Validación:** 80/20 train/test o barrido de  $k$ .
4. **Evaluación:** Métricas cuantitativas y análisis.

## Criterio del equipo

Cada grupo propone y justifica su problema particular en el paper y en la defensa.

# Entregables del proyecto: Artículo científico y código



## Pautas formales de entrega en formato académico

### Estructura del artículo (Formato\_Investigadores.doc)

1. **Título, autores y resumen (abstract):** Síntesis de 150 a 250 palabras.
2. **Introducción:** Contexto y motivación en Big Data.
3. **Metodología:** Datos, entorno de cómputo y enlace al código.
4. **Resultados analíticos:** Respuestas y gráficos de las 5 consultas.
5. **Modelado con Spark MLlib:** Consigna 6 (problema, pipeline y métricas).
6. **Evaluación experimental de tiempos:** Tabla de benchmarking y Speedup.
7. **Conclusiones:** Justificación técnica y arquitectural.
8. **Referencias bibliográficas.**

### Código fuente reproducible

- **Enlace público obligatorio:** Repositorio GitHub o carpeta compartida de Google Drive.
- **Notebooks documentados:** Código limpio, estructurado y reproducible de inicio a fin.
- **Instrucciones claras:** Especificar versiones de software y pasos de ejecución.

### Rigor metodológico

Todo resultado expuesto en el artículo debe derivarse directamente del código entregado.

### Defensa oral (máximo 10 slides)

- **Tiempo:** 12 minutos de exposición grupal + 3 minutos de preguntas del docente.
- **Estructura sugerida:**
  - S1: Portada (grupo, integrantes).
  - S2: Dataset y especificaciones de hardware.
  - S3-S5: Síntesis de las 5 consultas analíticas.
  - S6: Consigna 6 (planteo y pipeline de MLlib).
  - S7: Consigna 6 (métricas y evaluación de ML).
  - S8: Tabla de benchmarking (Pandas vs. Spark).
  - S9: Conclusiones y lecciones de arquitectura.
  - S10: Cierre y preguntas del docente.

### Dinámica del taller en clase 8

- Espacio intensivo de trabajo práctico en clase.
- Implementación de consultas y del pipeline de MLlib.
- Acompañamiento docente y resolución de dudas técnicas en vivo.
- Medición experimental de tiempos y optimización de planes de ejecución.

**¡Aprovechemos el taller para consolidar los experimentos!**



# ¿Preguntas o consultas iniciales?

Espacio para resolver dudas sobre el enunciado y la organización de los grupos